

Disk scheduling Algorithms manage how data is read from and written to a computer's hard disk.

The main goal is to determine in which disk head & write sequence all processes.
→ Reduce the time it takes to access data & improve overall s/m efficiency.

Characteristics of disk scheduling

Seek time → Time taken to move the disk arm to track where data is located.

Rotational latency — Time taken for the desired sector to rotate under the r/w head.

Transfer time → Time taken to actually read or write data

$\text{Disk Access time} = \text{Seek time} + \text{Rotational latency} + \text{transfer time.}$

Advantages of FCFS

- Every request gets a fair chance
- NO indefinite postponement

Disadvantages

- Does not try to optimize seek time
- May not provide best possible service

SSTF

- Advantages → Avg response time decreases
- Throughput increases

- | | Advantage | Disadvantage |
|---------------|---|---|
| <u>C-Scan</u> | → Eliminates starvation, better performance. | → Overhead to calculate seek time in advance |
| | → Disk arm moves one direction only | → Can cause starvation if it has a higher seek time as compared to incoming requests. |
| | → After reaching the end, it jumps to the beginning | → Extra head movement |
| | → Serves request in circular manner | → Increase seek time |
| | | → More overhead |

~~Look & Scan~~

SCAN

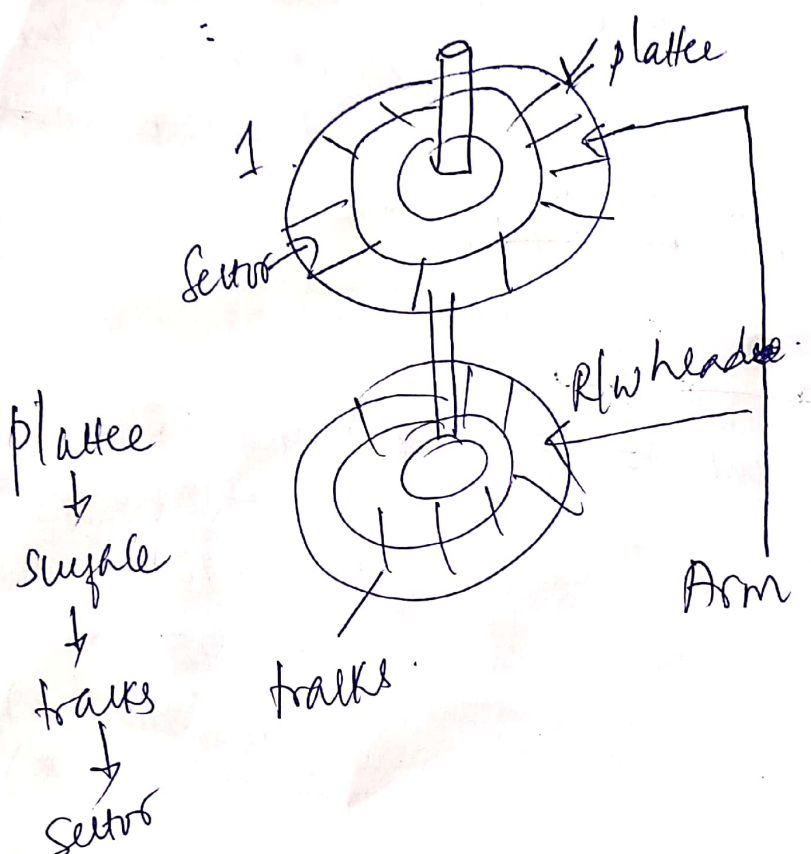
- High throughput
- Low variance of response time
- Avg response time.

- Head may move to disk end
- High waiting time for some requests
- New requests may wait longer

Disk Scheduling Algorithms.

- ✓ FCFS
- SSTF
- ⋮

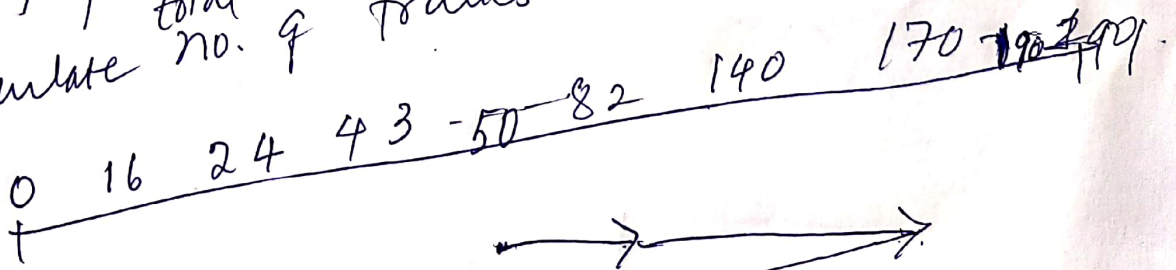
Goal is to minimize the seek time.
 Seek time → time taken to reach up to desired track.



Disk Access time = seek time + Rotational latency + Transfer time
 (waiting for sector to rotate under head)
 (Actual data transfer)

FCFS

A disk contains 200 tracks (0-199). Request queue contains track no. 82, 170, 43, 140, 24, 16, 190 respectively. Current pos'n of R/W head = 50. Calculate total no. of tracks movement by R/W head.



Total no. of tracks movement = $(82-50) + (170-82) + (170-43) + (140-43) + (140-24) + (24-16) + (190-16)$
 $= 32 + 88 + 127 + 97 + 116 + 8 + 174 = 642$

SSTF
[It gives optimal result]

[Need to go reverse]

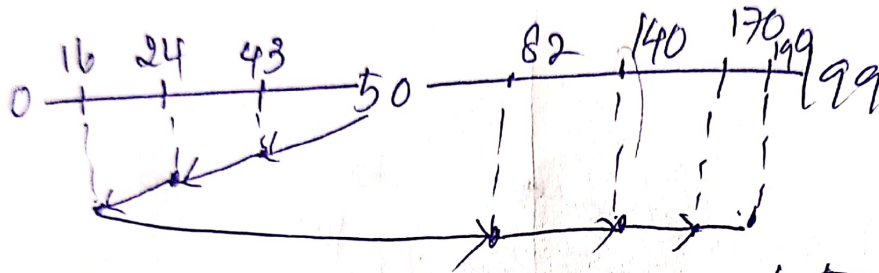
For the above problem calculate total no. of tracks movement by R/W head

using shortest seek time first?

if R/W head takes n s to move from one tracks to another then total

time taken = ?

$$208 \times 1 \text{ ns} = 208 \text{ ns}$$



$$\text{Total no. of tracks movement} = (50 - 16) + (199 - 50) + (24 - 16) + (82 - 16) + (140 - 82) + (170 - 140) + (199 - 170)$$

$$\text{or } (50 - 16) + (199 - 16)$$

can be done

$$14 + 174 = 208$$

problem is starvation / overhead.

tail head

Disk Scheduling

→ FCFS scheduling. (First come First served algorithm)

The simplest form of disk scheduling is, of course, the first come, first served (FCFS) algorithm. This algorithm is fair, but generally does not provide fastest service.

Ex: consider disk queue with requests for I/O to blocks on cylinders

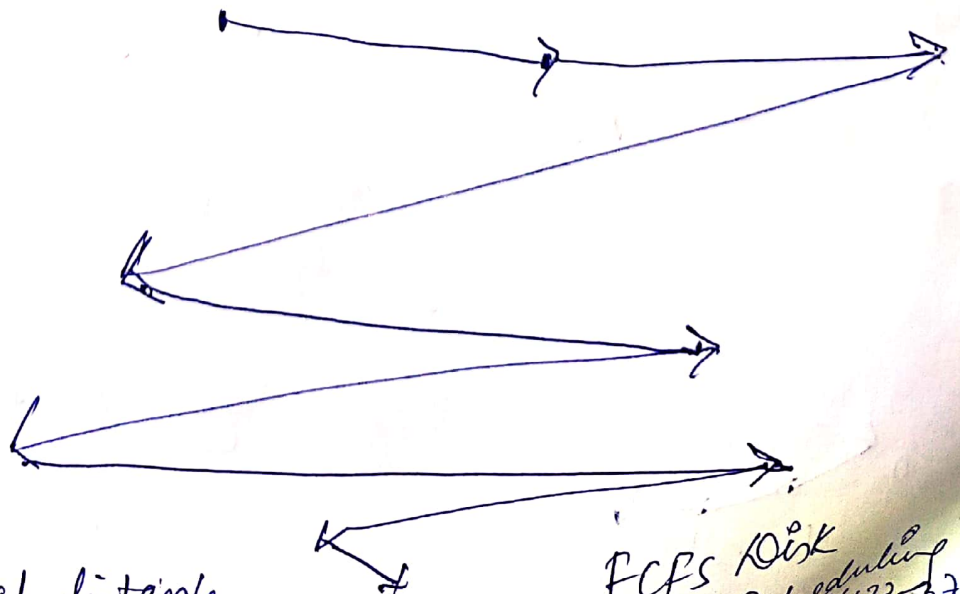
98, 183, 37, 122, 14, 124, 65, 67

Queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53.

0 14 37 53 65 67 98 122 124 183

Disk head is initially at cylinder 53. First it will move from 53 to 98 then to 183, 37, 122, 14, 124, 65, 67.



Total distance covered by disk head =

$$= (98 - 53) + (183 - 98) + (37 - 183) + (122 - 37) + (14 - 122) + (124 - 14) + (65 - 124) + (67 - 65)$$

FCFS Disk Scheduling

3) Scan algorithm

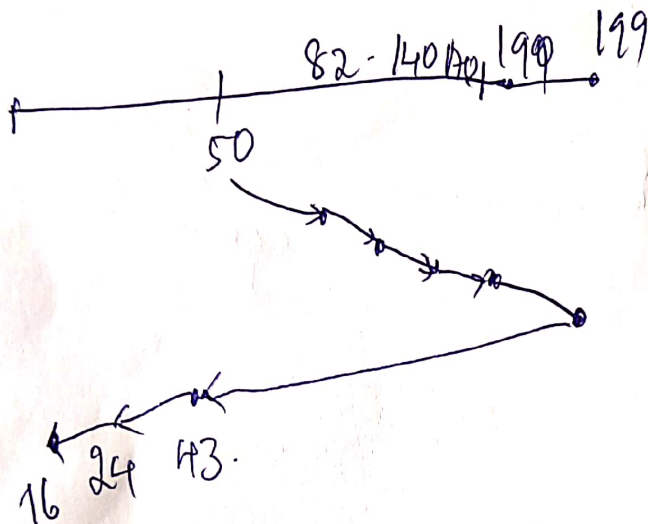
consider above problem
200 tracks (0 to 199)

[Move to
larger value
or smaller value
given in
request]

Request Queue

→ 82, 170, 43, 140, 24, 16, 199

current posn of R/W head = 50



[It will go
to end i.e. 199.]

[Direction
towards
larger
value]

$$(199 - 50) + (199 - 16) = 332$$

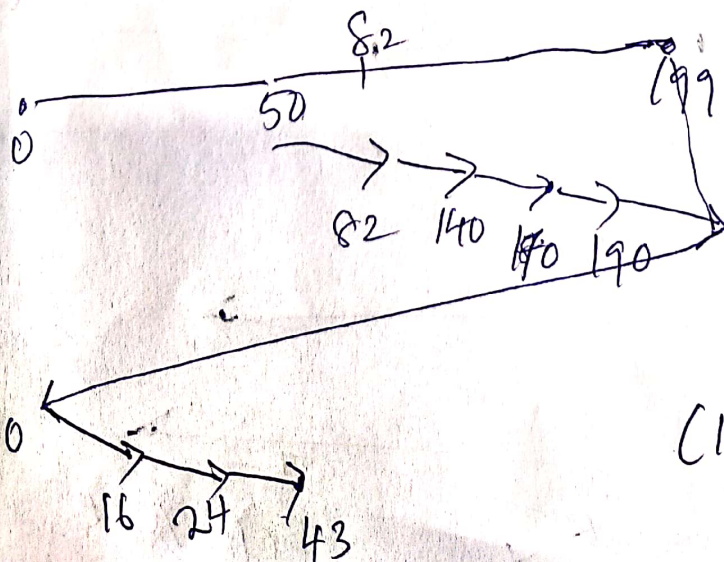
4) C-Scan algorithm

consider above problem

↙ circular scan

→ 82, 170, 43, 140, 24, 16, 199

R/W head
posn = 50



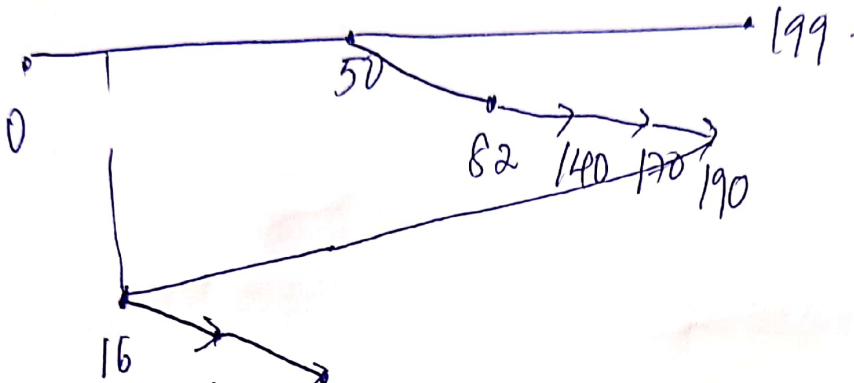
→ Direction towards
larger value

$$(199 - 50) + (199 - 0) + (43 - 0) = 391$$

C Look Algorithm in Disk Scheduling

consider same problem

82, 170, 43, 140, 24, 16, 190.



Direction towards large value.

In C-look disk scheduling, no seeking is required.

$$(190 - 50) + (190 - 16) + (24 - 16) + (43 - 24) + (43 - 16)$$

Look

Disk head looks ahead & goes only till the last request in that direction & then reverse.

$$= 341$$

Selection of Disk Scheduling Algo.

Given so many how do you choose

best algo.

SSTF increases performance over FCFS
SCAN & C-SCAN perform better but
place a heavy load on the disk.

Types of Requests are also imp.